// lab exam · practice set

Lab Exam Practice

Questions with full answers to try before the exam

A · React signup events **B · DOM manipulation** **C · MongoDB queries + map / filter / reduce**

HOW TO PRACTISE

Every question is followed by its answer slide

1

Read the question

Note what the examiner wants to see: the event, the object, the console output, the exact query result.

2

Try it yourself

Code it in VS Code, the browser console or mongosh. Give yourself 10 minutes before looking.

3

Compare and fix

Match your output with the answer slide. Then do the “Practice more” variations of each part.



React signup form

useState per field → e.target.value → userData → console and fetch

Level 1: 2 inputs · Level 2: 3 inputs · Level 3: 4 inputs + POST · Level 4: redirect

PART A · LEVEL 1 · QUESTION

Two inputs: fill in the blanks in App.js

```
import React, { useState } from 'react';

export default function App() {
  const [username, setUsername] = useState('');
  const [password, setPassword] = _______;

  const handleSignup = (e) => {
    ___________; // stop reload
    const userData = { _______ };
    console.log(userData);
  };

  return (
    <form onSubmit={_______}>
      <input type="text" value={username}
        onChange={(e) => setUsername(_______)} />
      <input type="password" value={password}
        onChange={_______} />
      <button type="submit">Signup</button>
    </form>
  );
}
```

Goal: type a name and password, click Signup → the console shows both in one object · 6 blanks

PART A · LEVEL 1 · ANSWER

The six blanks, filled

```
import React, { useState } from 'react';

export default function App() {
  const [username, setUsername] = useState('');
  const [password, setPassword] = useState('');

  const handleSignup = (e) => {
    e.preventDefault();
    const userData = { username, password };
    console.log(userData);
  };
  return (
    <form onSubmit={handleSignup}>
      <input type="text" value={username}
        onChange={(e) => setUsername(e.target.value)} />
      <input type="password" value={password}
        onChange={(e) => setPassword(e.target.value)} />
      <button type="submit">Signup</button>
    </form>
  );
}
```

Console: {username: 'riya', password: 'riya@123'} · e.target = the input box, e.target.value = what was typed

PART A · LEVEL 2 · QUESTION

Three inputs: add an email field on your own

Start from your Level 1 answer. No blanks this time: find the **three places** you must change, and print the data as **JSON** too.

?

Change 1

Where will the email value live?

?

Change 2

How does it get into userData?

?

Change 3

What does the user type into?

Think first: every new field needs the same three things

PART A · LEVEL 2 · ANSWER

One state, one key, one input

1 New state

```
const [email, setEmail] = useState('');
```

2 Add to userData

```
const userData = { username, email, password };
console.log(userData);
console.log(JSON.stringify(userData)); // JSON text
```

3 New input

```
<input type="email" value={email}
  onChange={(e) => setEmail(e.target.value)} />
```

Shorthand: { username, email } means { username: username, email: email }

PART A · LEVEL 3 · QUESTION

Four inputs, then POST the data with fetch

Fields: **username, number, email, password**. On Signup:

1. Add the **number** field the Level 2 way
2. Make handleSignup **async** and log userData
3. POST it to http://localhost:8000/api/auth/register with **fetch**
4. Log the server's reply; wrap it in **try / catch**
5. Mark every input **required**

The examiner checks

- 4 useState, 4 onChange
- e.preventDefault() first
- method, headers, body
- body is JSON.stringify(…)
- await on fetch and on .json()
- catch prints the error

No backend in the lab? The userData log still shows, and catch prints "Failed to fetch"

handleSignup with async / await

```
const [number, setNumber] = useState(''); // + username, email, password

const handleSignup = async (e) => {
  e.preventDefault();
  const userData = { username, number, email, password };
  console.log(userData);
  try {
    const response = await fetch('http://localhost:8000/api/auth/register', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(userData)
    });
    const responseData = await response.json();
    console.log(responseData);
  } catch (error) {
    console.error('Error:', error);
  }
};
```

async / await
await works only inside an async function.

Two awaits
One for fetch, one for .json(). Skip one: you log a Promise.

headers
Tells the server the body is JSON.

PART A · LEVEL 3 · ANSWER, THE FORM

Four inputs, the same pattern four times

```
return (
  <form onSubmit={handleSignup}>
    <input type="text" id="username" value={username}
      onChange={(e) => setUsername(e.target.value)} required />
    <input type="text" id="number" value={number}
      onChange={(e) => setNumber(e.target.value)} required />
    <input type="text" id="email" value={email}
      onChange={(e) => setEmail(e.target.value)} required />
    <input type="password" id="password" value={password}
      onChange={(e) => setPassword(e.target.value)} required />
    <button type="submit">Signup</button>
  </form>
);
```

4 fields = 4 × 3

Each needs a useState, a key in userData and an input.

required

The browser blocks submit while a box is empty.

Labels

In JSX write htmlFor, not for; className, not class.

PART A · LEVEL 4 · READ THE CODE, ANSWER THE VIVA

Redirect to login after a successful signup

```
import { useNavigate } from 'react-router-dom';
// inside the component
const navigate = useNavigate();

// inside try, after responseData
if (response.ok && responseData.redirectToLogin) {
  alert('Registration successful!');
  navigate('/login');
} else {
  console.error('Error registering user');
}
```

Q · What is response.ok?

true when the status is 200–299.

Q · When does else run?

Server replied, but with an error or no redirect flag.

Q · Server is off?

fetch throws, so catch runs, not else.

Q · useNavigate crashes?

Wrap App in <BrowserRouter> in main.jsx.

Ask the questions on the right before revealing the answers

PART A · CHECK YOUR OUTPUT

What the console shows, and common mistakes

```
{username: 'riya', number: '9876543210', email: 'riya@gmail.com', password: 'riya@123'}
Error: TypeError: Failed to fetch // only when no server is running
```

Mistake	What you see	Fix
onChange={setEmail(e.target.value)}	Error: e is not defined	Wrap it: (e) => setEmail(…)
value without onChange	You cannot type in the box	Add the onChange
Forgot e.preventDefault()	Page reloads, console clears	Call it first
body: userData	Server gets [object Object]	JSON.stringify(userData)
response.json() without await	Logs Promise {<pending>}	await response.json()

PART A · VIVA QUESTION

“Ten fields means ten useStates?”

One object state

One handler for all inputs; name must match the key.

```
const [userData, setUserData] = useState({ username: '', email: '' });
const handleChange = (e) =>
  setUserData({ ...userData, [e.target.name]: e.target.value });
<input name="email" value={userData.email} onChange={handleChange} />
```

FormData: no state

Reads every named input straight from the form.

```
const handleSignup = (e) => {
  e.preventDefault();
  const userData = Object.fromEntries(new FormData(e.target));
  console.log(userData);
};
```

All ways give the same userData · separate useStates (Levels 1–3) is the expected exam answer

PART A · PRACTICE MORE

Six variations the examiner can ask

#	Task	Key line
1	Add “Confirm password”; stop if different	if (password !== confirm) { alert('Passwords do not match'); return; }
2	Number must be exactly 10 digits	if (number.length !== 10) { alert('Enter 10 digits'); return; }
3	Clear all fields after submit	setUsername(''); setNumber(''); setEmail(''); setPassword('');
4	Show / hide password checkbox	type={show ? 'text' : 'password'} · onChange={(e) => setShow(e.target.checked)}
5	Live greeting under the form	Hello, {username || 'guest'}
6	Disable Signup until all are filled	disabled={!username || !number || !email || !password}

Checkboxes use e.target.checked, every other input uses e.target.value

B

DOM manipulation

One HTML file, one click listener, a visible “waah” change

PART B · QUESTION 2

The Mood Switcher card

Using only HTML and plain JavaScript, make one button change the page on every click:

1. The **background colour** changes
2. The **emoji** and the **message** change
3. A **click counter** goes up by one
4. After the last mood it **loops back** to the first

Use addEventListener("click"), getElementById, textContent and style.

Given index.html

```
<body>
  <div id="card">
    <h1 id="emoji">🙂</h1>
    <p id="msg">Click to change my mood!</p>
    <button id="btn">Change Mood</button>
    <p>Clicks: <span id="count">0</span></p>
  </div>
  <script src="script.js"></script>
</body>
```

Write script.js · answer on the next slide

PART B · ANSWER 2

script.js: an array of moods and one listener

```
const moods = [
  { emoji: "😄", text: "Happy!", color: "#FFD93D" },
  { emoji: "😎", text: "Cool!", color: "#6BCB77" },
  { emoji: "😴", text: "Sleepy...", color: "#4D96FF" },
  { emoji: "🤩", text: "Excited!", color: "#FF6B6B" }
];
let i = 0;
let clicks = 0;

document.getElementById("btn").addEventListener("click", () => {
  i = (i + 1) % moods.length; // next mood, loop to 0
  clicks++;
  document.getElementById("emoji").textContent = moods[i].emoji;
  document.getElementById("msg").textContent = moods[i].text;
  document.body.style.backgroundColor = moods[i].color;
  document.getElementById("count").textContent = clicks;
});
```

% moods.length makes the index go 1, 2, 3, 0, 1 … so it never runs past the array

Each click cycles through

- Happy · #FFD93D
- Cool · #6BCB77
- Sleepy · #4D96FF
- Excited · #FF6B6B

“Waah” tip

Add body { transition: background-color 0.5s; } in a style tag for a smooth colour fade.

PART B · QUESTION 2B WITH ANSWER

To-do list: create and remove elements

Type a task and click Add: a new list item appears. Click an item: it disappears. Empty input is ignored.

Given HTML

```
<input id="task" placeholder="Task">
<button id="add">Add</button>
<ul id="list"></ul>
```

Answer: script.js

```
const btn = document.getElementById("add");
const input = document.getElementById("task");
const list = document.getElementById("list");

btn.addEventListener("click", () => {
  if (input.value.trim() === "") return; // skip empty
  const li = document.createElement("li");
  li.textContent = input.value;
  li.addEventListener("click", () => li.remove()); // delete
  list.appendChild(li);
  input.value = ""; // clear the box
});
```

createElement → set textContent → appendChild: the three steps of adding anything to the page

PART B · CHEAT SHEET

The DOM methods you need in the exam

I want to…	Write
Find an element	document.getElementById("btn") · document.querySelector(".box")
Run code on click	el.addEventListener("click", () => { … })
Change the text	el.textContent = "Hello"
Change the HTML inside	el.innerHTML = "Hello"
Change one style	el.style.backgroundColor = "tomato"
Switch a CSS class on / off	el.classList.toggle("dark")
Add a new element	const li = document.createElement("li"); list.appendChild(li)
Read what was typed	input.value
Remove an element	el.remove()

PART B · PRACTICE MORE

Six small DOM tasks with a visible change

#	Task	Key line
1	Dark mode toggle button	document.body.classList.toggle("dark") + a .dark rule in CSS
2	Show / hide password	pwd.type = pwd.type === "password" ? "text" : "password"
3	Live character counter	box.addEventListener("input", e => count.textContent = e.target.value.length)
4	Random colour generator	"#" + Math.floor(Math.random() * 16777215).toString(16).padStart(6, "0")
5	A+ / A− font size buttons	size += 2; para.style.fontSize = size + "px"
6	Swap the image on click	img.src = "night.jpg"; img.alt = "Night view"

Every task: find the element → add a listener → change text, style, class or attribute



MongoDB queries

On the Fake Store products data · plus map, filter and reduce

PART C · SETUP

Load fakestoreapi.com/products into MongoDB

Import once (terminal)

```
# save the URL's JSON as products.json
mongoimport --db shop --collection products
  --file products.json --jsonArray

# open the shell
mongosh
use shop
db.products.countDocuments() // 20
```

Every document looks like this

```
{
  id: 1,
  title: "Fjallraven - Foldsack No. 1 ...",
  price: 109.95,
  description: "...",
  category: "men's clothing",
  image: "https://fakestoreapi.com/img/...",
  rating: { rate: 3.9, count: 120 }
}
```

20 products | electronics · 6 | women's clothing · 6 | men's clothing · 4 | jewelery · 4

Category names must be typed exactly as in the data: "jewelery" (not jewellery), "men's clothing"

PART C · LEVEL 1: FIND

Basic queries with answers

#	Question	Query	Answer
1	All electronics	db.products.find({ category: "electronics" })	6 products
2	Price under 50	db.products.find({ price: { $lt: 50 } })	9 products
3	Price from 100 to 200	db.products.find({ price: { $gte: 100, $lte: 200 } })	5 (ids 1, 6, 10, 11, 12)
4	Rating 4.5 or more	db.products.find({ "rating.rate": { $gte: 4.5 } })	6 products
5	Show only title and price	db.products.find({}, { title: 1, price: 1, _id: 0 })	20 docs, 2 fields
6	How many jewelery items?	db.products.countDocuments({ category: "jewelery" })	4

Nested field? Use dot notation in quotes: "rating.rate" · second argument of find() = projection

PART C · LEVEL 2: OPERATORS

AND, OR, IN, NOT and text search

#	Question	Query	Answer
7	Electronics AND rating above 4	db.products.find({ category: "electronics", "rating.rate": { $gt: 4 } })	2 (ids 11, 12)
8	Men’s OR women’s clothing	db.products.find({ category: { $in: ["men's clothing", "women's clothing"] } })	10 products
9	Price under 10 OR rating 4.8+	db.products.find({ $or: [{ price: { $lt: 10 } }, { "rating.rate": { $gte: 4.8 } }] })	5 (ids 7, 11, 12, 18, 19)
10	Title contains “jacket”	db.products.find({ title: { $regex: "jacket", $options: "i" } })	4 products
11	Everything except electronics	db.products.find({ category: { $ne: "electronics" } })	14 products

Two conditions in one { } = AND · $or takes an array of conditions · $options: "i" = ignore case

PART C · LEVEL 2: SORT, LIMIT, SKIP

Top-N and paging questions

#	Question	Query	Answer
12	Top 3 most expensive	db.products.find().sort({ price: -1 }).limit(3)	Samsung monitor 999.99 · John Hardy bracelet 695 · Acer monitor 599
13	3 most reviewed	db.products.find().sort({ "rating.count": -1 }).limit(3)	Rain Jacket 679 · Mens Cotton Jacket 500 · SanDisk SSD 470
14	Page 2 of the cheapest (5 per page)	db.products.find().sort({ price: 1 }).skip(5).limit(5)	15.99, 22.3, 29.95, 39.99, 55.99
15	All category names	db.products.distinct("category")	4 names

sort: 1 = ascending, −1 = descending · order is always sort → skip → limit

PART C · LEVEL 3: UPDATE AND DELETE

Change the data with $set and $inc

```
// Q16 change the price of product 1
db.products.updateOne({ id: 1 }, { $set: { price: 99.99 } })

// Q17 add 10 to every electronics price → matchedCount: 6
db.products.updateMany({ category: "electronics" }, { $inc: { price: 10 } })

// Q18 add a new field to all 20 products
db.products.updateMany({}, { $set: { inStock: true } })

// Q19 delete one product, then all rated below 2 → removes id 8 (1.9)
db.products.deleteOne({ id: 20 })
db.products.deleteMany({ "rating.rate": { $lt: 2 } })
```

Practise on a copy · reset anytime: run mongoimport again with --drop

PART C · LEVEL 4: AGGREGATE

Q20 · Count, average price and rating per category

```
db.products.aggregate([
  { $group: {
      _id: "$category",
      total: { $sum: 1 },
      avgPrice: { $avg: "$price" },
      avgRating: { $avg: "$rating.rate" }
  } },
  { $sort: { avgPrice: -1 } }
])
```

_id	total	avgPrice	avgRating
electronics	6	≈ 332.50	≈ 3.48
jewelery	4	≈ 221.00	3.35
men's clothing	4	≈ 51.06	3.70
women's clothing	6	≈ 26.29	≈ 3.68

mongosh prints more decimals; values here are rounded.

Q21 Total of all prices `{ $group: { _id: null, sum: { $sum: "$price" } } }` → **3240.92**

"$price" with a $ = the value of that field · _id: null = one group for the whole collection

PART C · LEVEL 4: PIPELINE

Q22 · Electronics under 200: title and price, cheapest first

```
db.products.aggregate([
  { $match: { category: "electronics", price: { $lt: 200 } } },
  { $project: { _id: 0, title: 1, price: 1 } },
  { $sort: { price: 1 } }
])
```

Result: 4 documents

WD 2TB Elements · 64

SanDisk SSD PLUS 1TB · 109

Silicon Power 256GB SSD · 109

WD 4TB Gaming Drive · 114

$match

keeps some documents, like JS **filter**

$project

reshapes every document, like JS **map**

$group

squeezes many into one, like JS **reduce**

Each stage feeds the next one, top to bottom

PART C · MAP, FILTER, REDUCE

Same questions, with JavaScript arrays

```
// get the array: in mongosh
const products = db.products.find().toArray();
// or in the browser console
const products = await (await fetch("https://fakestoreapi.com/products")).json();
```

Method	What it does	MongoDB twin	Example
filter	keeps items that pass a test	find() · $match	p => p.price < 50
map	changes every item, same length	$project	p => p.title
reduce	combines all items into one value	$group · $sum	(sum, p) => sum + p.price, 0

The old db.collection.mapReduce() is deprecated: use aggregate() in MongoDB and these array methods in JS

PART C · SMALL QUERIES IN JS

filter, map and reduce, one at a time

```
// filter → products under 50 (9 items)
const cheap = products.filter(p => p.price < 50);

// map → only their titles
const titles = cheap.map(p => p.title);

// reduce → total of all prices
const total = products.reduce((sum, p) => sum + p.price, 0);
console.log(total.toFixed(2)); // "3240.92"

// reduce → count per category
const perCat = products.reduce((acc, p) => {
  acc[p.category] = (acc[p.category] || 0) + 1;
  return acc;
}, {});
// { "men's clothing": 4, jewelery: 4, electronics: 6, "women's clothing": 6 }
```

PART C · MEDIUM QUERIES IN JS

Chain them: filter → sort → map → reduce

```
// average price of electronics → 332.50
const elec = products.filter(p => p.category === "electronics");
const avg = elec.reduce((s, p) => s + p.price, 0) / elec.length;

// titles rated 4.5+, best first → 6 titles
products.filter(p => p.rating.rate >= 4.5)
  .sort((a, b) => b.rating.rate - a.rating.rate)
  .map(p => p.title);

// 10% discount on everything (map returns new objects)
const sale = products.map(p => ({ ...p, price: +(p.price * 0.9).toFixed(2) }));

// most expensive with reduce → Samsung monitor, 999.99
const top = products.reduce((max, p) => (p.price > max.price ? p : max));
```

Same answers as the MongoDB queries: $match ≈ filter, $sort ≈ sort, $project ≈ map, $group ≈ reduce

PART C · TRY YOURSELF

Ten questions: write the query, then check

T1 How many products have 400 or more reviews?

T2 Which is the cheapest product, and its price?

T3 How many women’s clothing items cost under 30?

T4 How many products cost more than 100?

T5 What is the highest rating, and which products have it?

T6 How many titles start with “Mens”?

T7 What is the total price of all electronics?

T8 What is the average rating of all 20 products?

T9 JS: with reduce, count the products under 50.

T10 JS: with map, make { name, price } objects with the title in UPPERCASE.

Use the original data: re-import with --drop if you ran the update practice

PART C · ANSWERS T1–T5

Check your queries

#	Query	Answer
T1	db.products.countDocuments({ "rating.count": { $gte: 400 } })	7
T2	db.products.find().sort({ price: 1 }).limit(1)	Opna Women’s Short Sleeve, 7.95
T3	db.products.countDocuments({ category: "women's clothing", price: { $lt: 30 } })	4
T4	db.products.countDocuments({ price: { $gt: 100 } })	8
T5	db.products.find().sort({ "rating.rate": -1 }).limit(1) then db.products.find({ "rating.rate": 4.8 })	4.8 · ids 11 and 12 (Silicon Power SSD, WD 4TB drive)

T5 is a tie: limit(1) alone would hide the second product, so look up the value again

PART C · ANSWERS T6–T10

Check your queries

#	Query	Answer
T6	db.products.countDocuments({ title: { $regex: "^Mens" } })	3
T7	db.products.aggregate([{ $match: { category: "electronics" } }, { $group: { _id: null, total: { $sum: "$price" } } }])	≈ 1994.99
T8	db.products.aggregate([{ $group: { _id: null, avg: { $avg: "$rating.rate" } } }])	≈ 3.56
T9	products.reduce((n, p) => (p.price < 50 ? n + 1 : n), 0)	9
T10	products.map(p => ({ name: p.title.toUpperCase(), price: p.price }))	20 objects

^ in a regex means “starts with” · T9 gives the same 9 as Level 1, question 2

// before you submit

Exam-day checklist

React

- value + onChange on every input
- e.preventDefault() first
- userData = { … } logged
- JSON.stringify in the body

DOM

- script tag after the HTML
- ids spelled the same in both
- open the console for errors
- show the change live

MongoDB

- use shop first
- "jewelery" spelled as in data
- "rating.rate" in quotes
- every $ operator inside { }

All the best for the lab exam!